

Informe EP3: Observabilidad de agente de IA

Asignatura: Ingeniería de Soluciones con IA

Proyecto: Caso 3 - Aplicación RAG para evaluación de candidatos por competencias

Contexto: Salmones Camanchaca S.A.

Entrega: Evaluación Parcial N°3 - Observabilidad de agentes de IA

Introducción

El proyecto consiste en una aplicación web que apoya la preselección documental de candidatos mediante IA generativa, RAG y un flujo alternativo basado en agente LangChain. La solución permite seleccionar un anuncio laboral, analizar CV, deducir competencias requeridas, buscar evidencia documental, generar un ranking y producir una terna recomendada.

Para esta entrega se incorporó una capa de observabilidad sobre el agente de IA existente. La observabilidad permite comprender el comportamiento interno de un sistema a partir de sus salidas, principalmente métricas, registros y trazas. En el caso de un agente de IA, esto es especialmente importante porque sus resultados dependen de múltiples componentes: modelo LLM, prompts, recuperación RAG, servicios internos, herramientas del agente, fallback local y generación de reportes.

La implementación realizada busca responder tres preguntas centrales:

1. ¿Cómo se comporta el agente durante una ejecución?
2. ¿Dónde se producen errores, latencias o comportamientos de baja calidad?
3. ¿Qué acciones concretas pueden mejorar su desempeño, estabilidad y uso responsable?

La observabilidad se implementó de forma integrada dentro de la misma aplicación, sin requerir infraestructura externa. Esto permite que el prototipo académico registre métricas, genere un dashboard, guarde evidencia técnica y mantenga trazabilidad tanto en el flujo clásico como en el flujo con agente LangChain.

A. Implementación de métricas de observabilidad

La solución incorpora un servicio backend denominado `ObservabilityService`, encargado de construir un resumen observable de cada ejecución. Este servicio recibe el resultado del análisis, el

estado del job y los eventos registrados durante la ejecución. A partir de esa información genera un snapshot de observabilidad en formato JSON.

La observabilidad se aplicó sobre los dos modos principales del sistema:

- flujo clásico de análisis RAG;
- flujo alternativo con agente LangChain.

Esto es relevante porque el modo agente incorpora planificación, herramientas, memoria y trazabilidad adicional. Por tanto, no bastaba con observar solo el pipeline clásico: también era necesario observar el comportamiento del agente cuando coordina el proceso.

Métricas de rendimiento

Se implementaron métricas de latencia para estimar el tiempo requerido por el agente durante el análisis:

Métrica	Descripción	Utilidad
Latencia total	Tiempo completo desde el inicio hasta el cierre de la ejecución.	Permite evaluar la experiencia general del usuario.
Latencia promedio por candidato	Tiempo total dividido por la cantidad de CV evaluados.	Ayuda a estimar escalabilidad cuando aumenta el número de postulantes.
Latencia promedio por evaluación	Tiempo total dividido por la cantidad de cruces candidato-competencia.	Permite identificar si la evaluación por competencia es demasiado costosa.

Estas métricas permiten detectar cuellos de botella, especialmente cuando el agente realiza muchas llamadas al LLM o cuando el proveedor externo responde lentamente.

Métricas de uso del LLM y fallback

El sistema registra el comportamiento del modelo externo y del fallback local:

Métrica	Descripción
Llamadas LLM exitosas	Cantidad de respuestas correctas obtenidas desde el modelo.
Usos de fallback local	Cantidad de veces que el sistema recurrió a reglas locales o similitud semántica.
Errores LLM	Fallos registrados durante llamadas al proveedor externo.
Tasa de éxito LLM	Porcentaje de eventos resueltos correctamente por el modelo.
Tasa de fallback	Porcentaje de eventos resueltos mediante respaldo local.

Métrica	Descripción
Tasa de error	Porcentaje de eventos que presentaron errores.

Estas métricas permiten evaluar dependencia del proveedor externo y resiliencia del sistema. Si el uso de fallback aumenta, puede indicar límites de solicitudes, errores temporales o problemas de configuración.

Métricas de calidad de evidencia

Como el sistema evalúa candidatos a partir de evidencia documental, se agregó una métrica de calidad basada en los puntajes de evidencia por competencia.

Métrica	Descripción
Evidencia promedio	Promedio del puntaje de evidencia observado.
Evidencia débil o no evidenciada	Proporción de evaluaciones con bajo respaldo documental.
Evidencia clara o fuerte	Proporción de evaluaciones con respaldo documental suficiente.

Esta métrica no reemplaza una validación humana, pero entrega una aproximación útil sobre la calidad del análisis. Si muchas competencias quedan con evidencia débil, el problema puede estar en el CV, en el texto extraído, en la consulta RAG o en la definición de competencias.

Métricas de uso de recursos

No se implementó medición directa de CPU, memoria ni costo monetario exacto por tokens. Sin embargo, sí se incorporaron métricas operativas asociadas al consumo del agente:

- cantidad de llamadas al LLM;
- cantidad de evaluaciones realizadas;
- uso de fallback local;
- errores del proveedor externo;
- latencia total y por evaluación.

Estas métricas funcionan como aproximación práctica al uso de recursos en un prototipo académico. A mayor cantidad de llamadas y mayor latencia, mayor será el costo operacional estimado del análisis.

B. Análisis de registros y trazabilidad

La trazabilidad se implementó mediante una bitácora de eventos por ejecución. Cada análisis queda asociado a un `trace_id`, lo que permite relacionar la ejecución visible en el frontend con el JSON de observabilidad y con el reporte generado.

Los eventos registran pasos relevantes del flujo, tales como:

- inicio de ejecución;
- selección del modelo;
- preparación del anuncio laboral;
- deducción de competencias;
- lectura de CV;
- construcción del índice RAG;
- evaluación de competencias;
- uso de LLM o fallback;
- generación de ranking;
- generación de reportes;
- finalización del análisis.

En el modo agente, la trazabilidad se complementa con `agent_trace`, que incluye:

- framework utilizado;
- tipo de agente;
- planificación generada;
- herramientas declaradas;
- plan de ejecución;
- decisiones adaptativas;
- herramientas ejecutadas;
- memoria de corto y largo plazo.

Esta información permite comprender no solo el resultado final, sino también el proceso seguido por el agente para llegar a dicho resultado.

Puntos críticos identificables mediante logs

El análisis de registros permite identificar áreas de mejora sin depender únicamente de la percepción del usuario. Los principales puntos críticos observables son:

Punto crítico	Señal observable	Impacto posible
Latencia alta	Aumento del tiempo por evaluación.	Menor escalabilidad y peor experiencia de uso.
Rate limit del proveedor	Errores asociados a exceso de solicitudes.	Esperas prolongadas o uso de fallback.
Alta tasa de fallback	Muchas evaluaciones resueltas localmente.	Resultado menos preciso que con LLM.
Baja evidencia documental	Muchas competencias sin respaldo suficiente.	Ranking menos confiable.

Punto crítico	Señal observable	Impacto posible
Ranking estrecho	Diferencia baja entre candidatos.	Requiere revisión humana más cuidadosa.

La trazabilidad permite transformar estos problemas en datos verificables. Por ejemplo, una demora no queda como una percepción general, sino como una métrica de latencia asociada a una ejecución y a eventos concretos.

C. Desarrollo de dashboard de monitoreo

Se desarrolló un dashboard de observabilidad integrado en el frontend de la aplicación. Aunque la pauta permite herramientas como Grafana, Kibana, Streamlit o PowerBI, en este caso se optó por un dashboard propio en HTML, CSS y JavaScript, porque la aplicación ya cuenta con frontend funcional y puede mostrar las métricas inmediatamente al finalizar cada análisis.

El dashboard muestra información organizada en tarjetas y paneles:

Sección del dashboard	Contenido
Métricas principales	Latencia, evaluaciones, éxito LLM, fallback, errores, evidencia y ranking.
Anomalías detectadas	Alertas generadas por reglas de observabilidad.
Recomendaciones automáticas	Sugerencias técnicas derivadas de las métricas.
Uso responsable	Alcance documental, revisión humana y exclusión de variables sensibles.
Trazabilidad técnica	Trace ID y archivo JSON asociado.

La decisión de construir un dashboard propio se justifica porque permite observar el comportamiento del agente dentro del flujo real de uso. Además, evita agregar infraestructura externa innecesaria para un prototipo académico.

Figura 1. Dashboard de observabilidad implementado en el frontend.
Insertar captura del panel “Dashboard de observabilidad” mostrando tarjetas de métricas, anomalías, recomendaciones y uso responsable.

Figura 2. Bitácora y trazabilidad del agente.
Insertar captura de la barra de progreso o de la sección “Trazabilidad del agente” mostrando eventos, herramientas y planificación.

Desde el punto de vista técnico, el dashboard consume el campo `observability` devuelto por el backend. Este campo contiene el snapshot generado por `ObservabilityService`, por lo que la

interfaz no calcula métricas por su cuenta: solo visualiza datos preparados por el backend.

D. Recomendaciones técnicas para optimizar el agente

A partir de las métricas y la trazabilidad implementadas, se definieron recomendaciones prácticas para mejorar el desempeño del agente.

1. Mantener fallback rápido ante rate limit

Cuando un proveedor externo responde con error 429 Too Many Requests, insistir con varios reintentos puede aumentar mucho la latencia. Por eso se implementó una configuración de fallback rápido:

```
LLM_FAIL_FAST_ON_RATE_LIMIT=true
```

Con esta opción, el sistema activa fallback local ante rate limit, evitando esperas prolongadas. Esta recomendación mejora la resiliencia del agente y permite que el análisis continúe aunque el proveedor externo no esté disponible temporalmente.

2. Reducir llamadas innecesarias al LLM

La latencia del agente está muy relacionada con la cantidad de llamadas al modelo. Por ello, se recomienda mantener controlado el número de competencias generadas y evitar enviar evidencia excesivamente larga al LLM.

Una estrategia adecuada es conservar entre 4 y 8 competencias por anuncio, limitar los fragmentos enviados desde RAG y priorizar evidencia de mayor similitud semántica. Esto reduce tiempo de respuesta y costo operacional.

3. Usar métricas de evidencia para revisar calidad documental

Si la proporción de evidencia débil es alta, no necesariamente significa que el agente falló. También puede indicar que el CV no contiene información suficiente para respaldar las competencias del cargo. Por eso, la métrica debe interpretarse como señal de revisión, no como decisión automática.

La recomendación es usar esta métrica para que una persona revise manualmente los casos con baja evidencia antes de aceptar o descartar candidatos.

4. Mantener revisión humana obligatoria

El sistema entrega apoyo documental, pero no debe tomar decisiones automáticas de contratación. El ranking y la terna recomendada deben ser interpretados como una ayuda para ordenar

antecedentes, no como una decisión final.

Esta recomendación es especialmente importante porque los documentos pueden estar incompletos, el OCR puede extraer texto imperfecto y el modelo puede interpretar evidencia de manera limitada.

5. Guardar trazas y reportes por ejecución

Se recomienda mantener la generación de archivos JSON y Markdown por ejecución. Esto permite auditar el proceso, revisar resultados anteriores y comparar el comportamiento del sistema bajo distintos anuncios, candidatos o modelos.

6. Considerar una herramienta externa en una fase futura

Para un entorno productivo, sería recomendable integrar la observabilidad con herramientas especializadas como Grafana, OpenTelemetry o un stack de logs. En el prototipo actual, el dashboard integrado es suficiente para evidenciar las métricas principales, pero una solución productiva requeriría almacenamiento histórico, alertas y visualizaciones comparativas.

E. Seguridad, privacidad y uso responsable

La observabilidad implementada también incluye un bloque de uso responsable. Esto permite que el dashboard y los reportes recuerden explícitamente el alcance del sistema.

El sistema debe cumplir los siguientes criterios:

- funcionar solo como apoyo documental;
- requerir revisión humana final;
- no contratar ni descartar automáticamente candidatos;
- evaluar solo evidencia relacionada con el cargo;
- excluir variables sensibles.

Variables excluidas del análisis:

- edad;
- género;
- nacionalidad;
- estado civil;
- fotografía;
- dirección exacta;
- datos familiares;
- religión;
- situación médica;

- opiniones políticas.

Bases válidas de evaluación:

- formación académica;
- experiencia laboral;
- conocimientos técnicos;
- certificaciones;
- funciones realizadas;
- competencias relacionadas con el cargo.

Este enfoque reduce el riesgo de sesgos y refuerza que la aplicación debe ser usada como herramienta de apoyo para profesionales responsables del proceso de selección.

F. Archivos implementados y modificados

La observabilidad se implementó mediante cambios en backend, frontend y reportes.

Archivos nuevos

Archivo	Función
app/backend/app/services/observability_service.py	Construye métricas, anomalías, recomendaciones y snapshots JSON de observabilidad.
app/backend/outputs/observability/	Carpeta donde se guardan los archivos JSON de observabilidad por ejecución.

Archivos modificados

Archivo	Cambio principal
app/backend/app/main.py	Integra observabilidad en flujo clásico y flujo con agente LangChain.
app/backend/app/config.py	Agrega configuración para fallback rápido ante rate limit.
app/backend/app/services/llm_client.py	Ajusta manejo de errores 429 y fallback local.
app/backend/app/services/report_service.py	Incluye sección de observabilidad en reportes Markdown.

Archivo	Cambio principal
app/backend/app/models/schemas.py	Agrega campo observability en la respuesta del análisis.
app/frontend/index.html	Agrega sección “Dashboard de observabilidad”.
app/frontend/app.js	Renderiza métricas, anomalías, recomendaciones y uso responsable.
app/frontend/styles.css	Agrega estilos visuales para el dashboard.
readme.md	Documenta observabilidad, archivos modificados y cumplimiento EP3.

G. Cumplimiento de indicadores de la pauta

Indicador	Cumplimiento
IE1	Se implementan métricas de calidad, errores, uso de LLM, fallback y evidencia documental.
IE2	Se mide latencia total, latencia por candidato y latencia por evaluación.
IE3	Se registran eventos, bitácora y trazabilidad mediante <code>trace_id</code> y <code>agent_trace</code> .
IE4	Se identifican anomalías como fallback alto, baja evidencia, latencia alta y ranking estrecho.
IE5	Se implementa dashboard visual integrado en el frontend.
IE6	Se incorporan criterios de seguridad, privacidad y uso responsable.
IE7	Se proponen recomendaciones técnicas basadas en métricas y trazabilidad.
IE8	El informe considera capturas del dashboard y visualizaciones de trazabilidad.
IE9	La documentación se redacta con lenguaje técnico, claro y estructurado.

Conclusión

La implementación de observabilidad permite que la aplicación deje de ser solo un sistema que entrega resultados y pase a ser una solución auditable. Ahora el agente no solo genera competencias, evaluaciones, ranking y reportes, sino que también informa cómo se comportó durante la ejecución.

La solución incorpora métricas de latencia, uso del LLM, fallback, errores, calidad de evidencia, anomalías, recomendaciones y uso responsable. Además, la observabilidad funciona tanto en el

flujo clásico como en el flujo con agente LangChain, lo que permite monitorear el comportamiento del agente real de la aplicación.

El dashboard integrado, los reportes Markdown y los archivos JSON de observabilidad entregan evidencia suficiente para analizar desempeño, detectar áreas de mejora y justificar recomendaciones técnicas. Para una etapa futura, la solución podría ampliarse con herramientas externas de monitoreo, almacenamiento histórico y alertas automáticas.

Referencias

FastAPI. (2026). *FastAPI documentation*. <https://fastapi.tiangolo.com/>

GitHub. (2026). *GitHub Models documentation*. <https://docs.github.com/github-models>

Grafana Labs. (2026). *Dashboards: Grafana documentation*.
<https://grafana.com/docs/grafana/latest/visualizations/dashboards/>

LangChain. (2026). *Agents documentation*.
<https://docs.langchain.com/oss/python/langchain/agents>